ChatGPT

# 영어 시험 자동 생성 및 검수 시스템 (2026 버전)

## 1. 설계 목표

- **대상과 기능**: Phonics 수준에서 한국 고등학교 내신까지 다양한 영어 평가에 적용할 수 있는 영어 시험지 자동 생성 및 검수 시스템을 설계한다. 객관식 비중이 90%, 주관식 비중이 10%인 시험지를 생성하고, 사람이 확인해야 하는 경우를 최소화하여 **출제자(교사)**, **학부모·학생** 그리고 **검수자**가 사용할 수 있도록 한다.
- **성능 기준**: 최신 대형 언어 모델(LLM)과 선언적 프레임워크를 이용해 인간 수준의 문항 품질을 달성한다. 시스템은 생성·필터링·평가·최적화의 **4단계 파이프라인**으로 구성되며, 추론 성능·비용 효율·재현성·안정성을 모두 고려한다.
- **확장성**: 멀티 에이전트 프레임워크와 DSPy 3.0을 사용하여 모듈화된 코드를 작성하고 자동으로 최적화한다. 다양한 LLM을 교체하며 성능을 비교할 수 있도록 설계한다.

## 2. 최신 모델 및 기술 스택 (2026 년 기준)

### 2.1 대형 언어 모델 (LLM)

| 역할 | 모델 | 특징 | 근거 |
|---|---|---|---|
| **메인 생성 (주 난이도)** | **GPT-5.2** (OpenAI) | OpenAI의 플래그십 코드·에이전트용 모델로서 최대 **40만 토큰**의 컨텍스트 창과 **12만 8천 토큰**의 출력 한도를 가진다. `reasoning.effort` 매개변수로 none부터 xhigh까지 추론 난이도를 조절할 수 있으며, API를 통해 텍스트, 이미지 입력과 코드 인터프리터·웹 검색 도구를 제공한다 [1]. | OpenAI 문서에서 GPT-5.2가 40만 토큰 컨텍스트 창을 제공하고 agentic 작업에 최적화된 모델임을 확인했다 [1]. |
| **장문 지문 / 멀티모달** | **Gemini 3.1 Pro** (Google DeepMind) | 2026 년 2월 발표된 최신 프로 모델로, 100만 토큰의 컨텍스트 창과 6만4천 토큰 출력, **텍스트·음성·이미지·동영상·코드** 입력을 지원한다. Gemini 3.1 Pro는 Gemini 3 Pro 기반이며 대규모 멀티모달 데이터 세트를 이해할 수 있으며, Google이 제공하는 가장 진보된 모델로 평가된다 [2]. | 모델 카드에서 Gemini 3.1 Pro가 natively multimodal 모델이며 최대 1M 컨텍스트 창을 갖고 다양한 입력 유형을 처리한다는 것을 확인했다 [2]. |
| **평가 및 채점** | **Claude Sonnet 4.6 / Opus 4.6** (Anthropic) | Claude 4.6 시리즈는 **200 K 토큰 컨텍스트**를 기본으로 지원하고 **1 M 토큰 컨텍스트 옵션**을 베타로 제공한다. `extended thinking`과 `adaptive thinking`을 통해 어려운 문제에서 더 오래 추론하도록 조절할 수 있으며, code execution과 web fetch 도구도 지원한다 [3]. | Claude 4.6 모델 문서에서 Sonnet 4.6과 Opus 4.6이 200 K 컨텍스트(베타 옵션으로 1 M)와 extended thinking·adaptive thinking을 지원한다고 명시했다 [3]. |

| 역할 | 모델 | 특징 | 근거 |
|---|---|---|---|
| **비용 효율·1차 필터링** | **Llama 4 Scout / Maverick** (Meta) | Meta가 공개한 Llama 4 Scout 모델은 **10 M 컨텍스트 창**을 지원하는 개방형 멀티모달 모델이다. 단일 H100 GPU에서도 추론이 가능하며, 긴 문서 분석과 비용 효율성을 제공한다 [4]. | Meta 공식 사이트에서 Llama 4 Scout가 10 M 토큰 컨텍스트를 갖는 natively multimodal 모델임을 확인했다 [4]. |
| **개방형 모델(선택)** | **Moonshot Kimi K2 / DeepSeek V3** 등 | 대용량 MoE 구조로 비용 대비 성능을 제공하나, 시험 제작에서는 세부 제어가 어렵기 때문에 필터링 또는 백업 용도로만 활용한다. | Pluralsight의 2026년 모델 비교에서 Moonshot Kimi K2가 중국의 1조 매개변수 MoE 모델임을 언급하고 Llama 4 Scout가 10M 컨텍스트를 제공해 긴 문서 분석에 유리하다고 설명한다 [5]. |

**선정 전략**

1. **메인 생성**: GPT-5.2는 reasoning.effort 옵션으로 문제 유형에 따라 추론 강도를 조절할 수 있어 메인 생성에 적합하다. 양질의 텍스트 생성과 코드 기반 도구 호출이 가능하다 [1].
2. **멀티모달/장문**: Gemini 3.1 Pro는 1M 컨텍스트 창과 멀티모달 이해 능력으로 긴 지문과 멀티미디어 기반 질문 생성에 사용한다 [2].
3. **평가 및 채점**: Claude Sonnet 4.6과 Opus 4.6은 extended thinking·adaptive thinking 기능과 200K(1M beta) 컨텍스트를 제공하여 적절한 채점 모델로 선택한다 [3].
4. **비용 효율**: Llama 4 Scout/Maverick는 대용량 컨텍스트와 오픈 라이선스를 갖추어 1차 필터링이나 세분화된 작업에 활용한다 [4].

## 2.2 멀티 에이전트 프레임워크

2026 년에는 다양한 오픈소스 에이전트 프레임워크가 등장하였다. AIMultiple의 2026 년 비교에서 다음과 같은 장단점을 확인했다:

- **LangGraph**는 DAG(Directed Acyclic Graph) 구조로 작업을 정의해 **최저 지연 시간**을 달성하며, 복잡한 워크플로에서 효율적이다 [6].
- **Swarm**은 LLM 호출을 최소화하는 특수화된 도구 에이전트를 사용하여 토큰 사용량을 줄인다 [6].
- **CrewAI**는 역할 기반 멀티 에이전트 아키텍처를 제공해 상태 관리를 체계적으로 수행하며 생산 환경에 적합하다 [7].
- **LangChain**은 체인 중심 설계로 사용이 간단하지만, 멀티 에이전트 모드에서 토큰 비용과 지연이 증가하는 단점이 있다 [8].

**프레임워크 선택**: 시험 시스템에서는 **LangGraph**를 기본 선택으로 하되, 특정 역할 기반 작업에는 **CrewAI**를, 가벼운 실험에는 **Swarm**을 사용할 수 있도록 모듈화한다.

### 2.3 DSPy 3.0 및 MIPROv2

DSPy는 Stanford 연구진이 개발한 선언적 LLM 프로그래밍 프레임워크로 2025 년 8월 **DSPy 3.0**이 발표되었다. 주요 특징은:

- **프로그램을 코드로 정의**: 문자열 프롬프트 대신 **Signature**와 **Module**을 사용하여 입력·출력 형식을 선언한다 [9].
- **모듈화된 추론 전략**: `ChainOfThought`, `ReAct`, `MultiChainComparison` 등 알고리즘을 모듈화해 사용할 수 있다 [10].
- **컴파일러와 최적화**: DSPy 3.0은 **MIPROv2**, **GRPO**, **SIMBA** 등 최적화기를 제공해 예제와 지시어를 자동으로 생성·조정한다 [11]. 특히 MIPROv2는 지시어와 few-shot 예제를 동시 최적화하며, Bayesian Optimization으로 탐색한다.
- **멀티모달 지원**: DSPy 3.0은 텍스트뿐 아니라 이미지와 오디오 등 복합 타입을 처리할 수 있도록 개선되었다 [12].
- **스케일러빌리티**: 스레드 안전, 비동기 지원, usage tracking 기능을 통해 고병행성을 지원한다 [13].

따라서 본 시스템은 **DSPy 3.0**을 코어 오케스트레이터로 채택한다.

### 2.4 임베딩 및 벡터 검색

- **OpenAI** `text-embedding-3-large`: 영문·비영문 모두에서 뛰어난 성능을 보여 검색, 클러스터링, 분류 등에 사용된다 [14].
- **Cohere Embed v3 Multilingual**: 100 개 이상의 언어를 지원하는 대표적 멀티모달 임베딩 모델로 검색·분류·군집화에 적합하다 [15].
- 필요 시 **Llama Index** 또는 **Qdrant** 등 오픈소스 벡터 DB를 이용해 문항 유사도 검색 및 중복 방지를 수행한다.

### 2.5 프론트엔드 및 기타 라이브러리

- **Next.js 16**: 2025년 10월 출시된 최신 버전으로 캐시 컴포넌트, DevTools MCP 지원, Turbopack 안정화 및 React 19.2를 포함해 AI 개발에 최적화되어 있다 [16].
- **Tailwind CSS**와 **ShadCN/UI**: UI 구성요소를 빠르게 개발하기 위해 사용한다.
- **Litellm**: 다양한 LLM API를 통합하여 라우팅과 페일오버를 가능하게 한다.

## 3. 전체 아키텍처 설계

### 3.1 시스템 구성도

```
[Front-end]
  ├─ Teacher Dashboard (Next.js 16)
  ├─ Student Portal (Next.js 16)
  ├─ Admin/QA Dashboard

[API Gateway]
  ├─ Authentication (JWT/OAuth2)
  ├─ Rate Limiting
  └─ Routing to services
```

```
[Core Orchestrator]
  ├─ DSPy 3.0 Pipeline Engine
  ├─ LLM Router (LiteLLM)
  ├─ Multi-Agent Framework (LangGraph & CrewAI)
  ├─ Evaluation Engine (Answer Matching & PRM)
  └─ Uncertainty Scorer (Semantic Energy)

[LLM Providers]
  ├─ OpenAI (GPT-5.2, GPT-5 mini)
  ├─ Google (Gemini 3.1 Pro)
  ├─ Anthropic (Claude 4.6 Sonnet/Opus)
  ├─ Meta (Llama 4 Scout/Maverick)

[Data Layer]
  ├─ PostgreSQL 16 (metadata & logs)
  ├─ Qdrant or FAISS (vector search)
  ├─ Redis (cache & task queue)
  ├─ Object Storage (e.g., AWS S3 for training data)

[Monitoring & Observability]
  ├─ Langfuse / OpenTelemetry for tracing
  ├─ Prometheus & Grafana for metrics
  ├─ Cost Tracker (per-model token usage)

[Infrastructure]
  ├─ Cloud provider: AWS (ECS Fargate), GCP (Cloud Run) 혹은 Azure (AKS)
  ├─ CI/CD: GitHub Actions, Terraform
```

### 3.2 단계별 파이프라인

1. **문항 생성 (Generation)**
2. **Direct prompting**: GPT-5.2 또는 Gemini 3.1 Pro에 간결한 역할 프롬프트를 전달해 Phonics~고교 내신 수준 문항을 생성한다. Chain-of-Thought 강제를 피하고 `reasoning.effort` 값은 `medium`으로 설정하여 효율과 품질을 균형 있게 유지한다 [1].
3. **Best-of-N 샘플링**: 동일 지문에 대해 N개의 후보 문항을 병렬 생성한다(N=5~10). 이를 위해 Celery/Redis 큐를 사용해 비동기 태스크로 분산한다.

4. **DSPy Module**: 생성 서명(`GenerateQuestion`)을 정의하고 `dspy.Predict` 모듈로 래핑한다. MIPROv2 옵티마이저가 지시어와 few-shot 예제를 자동으로 조합한다 [11].

5. **불확실성 필터링 (Filtering)**

6. **Semantic Energy**: 모델 마지막 은닉층 임베딩을 여러 번 샘플링 후 클러스터링하여 분산(에너지) 값을 계산한다. 높은 에너지는 모델의 확신이 낮음을 의미하므로 해당 문항은 인간 검수 대기열로 라우팅한다. 구현 시 여러 샘플의 임베딩 분산과 엔트로피를 결합한 근사식을 사용한다.

7. **1차 필터 모델**: 비용 절감을 위해 Llama 4 Scout 또는 Kimi K2로 간단한 오류 여부를 판단한다. 임계값을 넘는 경우에만 고가 모델로 재검토한다.

8. **정밀 평가 (Evaluation)**

9. **Answer Matching**: Claude 4.6 또는 GPT-5.2를 심판으로 사용하여 객관식 보기 없이 문제와 지문만 제공하고 자유 응답을 생성한다. 그 후 원래 정답과 의미적 유사도를 측정하여 복수 정답 여부나 보기가 잘못된 문항을 걸러낸다. TRM/CompassVerifier와 같은 최신 reward model을 적용해 신뢰도를 높인다 [17].

10. **Process Reward Model (PRM)**: PRM은 중간 추론 단계까지 평가하는 모델이다. Claude Opus 4.6의 `adaptive thinking`을 활용하여 논리적 일관성을 분석하고 각 단계에 점수를 부여한다. 점수가 낮은 문항은 폐기하거나 재생성한다.

11. **지속적 최적화 (Continuous Optimization)**

12. **HITL 피드백**: Semantic Energy나 PRM에서 낮은 점수를 받은 문항만 검수자가 검토한다. 검수자가 수정한 데이터를 DSPy 3.0 파이프라인의 학습 세트에 지속적으로 반영하여 옵티마이저가 재컴파일하도록 한다.
13. **자동 재학습**: MIPROv2가 주기적으로 새로운 trainset을 수집해 지시어와 퓨샷 예제 가중치를 조정한다. 또한 GRPO(연속 적 강화)나 SIMBA(사용자 정의 피드백 학습)와 같은 DSPy 3.0 추가 옵티마이저를 도입하여 성능을 높인다 [11].

# 4. 구체적인 기술 스택 및 설정

## 4.1 백엔드

- **언어/프레임워크**: Python 3.11, **FastAPI**(비동기 지원), **SQLAlchemy**(ORM), **Celery** + **Redis**(비동기 태스크).
- **DSPy**: `dspy-ai` 패키지 (>=3.0); `torch`와 `pydantic` 의존성 포함. `MIPROv2`와 `GRPO` 옵티마이저를 활성화한다.
- **LiteLLM**: 여러 LLM API를 라우팅하기 위해 `litellm`(>=1.20)을 설치한다. `litellm.yaml` 파일에 각 모델과 API 키를 설정한다.
- **LLM API 라이브러리**: `openai`, `anthropic`, `google-generativeai`, `cohere` 등.
- **벡터 DB**: `qdrant-client`(>=1.7) 또는 `faiss-cpu`를 사용해 벡터 저장소를 구축한다.
- **로깅/모니터링**: `langfuse`와 `opentelemetry-sdk`, `prometheus_client`를 설치하여 추적과 비용 모니터링을 구현한다.
- **패키지 설치 예**:

```
pip install dspy-ai==3.* litellm fastapi uvicorn psycopg[binary] sqlalchemy
alembic \
  qdrant-client langfuse opentelemetry-sdk prometheus-client openai anthropic \
  google-generativeai cohere redis celery
```

## 4.2 프론트엔드

- **Next.js 16**: React 19.2와 호환되며, **Cache Components**와 **Partial Pre-Rendering** 지원으로 성능을 높인다 [16].

- **Tailwind CSS**, **ShadCN/UI**, **React Query**, **Zustand**(상태 관리), **Socket.IO**(실시간 상태 업데이트).
- **국제화**: 한국어 UI와 영어/한국어 문제 출력을 지원하기 위해 `next-i18next`를 사용한다.

### 4.3 인프라 및 배포

- **클라우드**: AWS (ECS Fargate), GCP (Cloud Run) 또는 Azure (AKS) 중 선택한다. 모델 호출량에 따라 GPU 인스턴스를 사용하거나 클라우드 LLM API만 호출한다.
- **IaC**: Terraform으로 VPC, ECS 클러스터, RDS(PostgreSQL 16), ElastiCache(Redis), S3 버킷, Qdrant 인스턴스를 설정한다.
- **CI/CD**: GitHub Actions를 이용해 코드 린트, 테스트, 빌드, 배포를 자동화한다.
- **보안**: JWT 인증, HTTPS(SSL/TLS), Rate limit(예: 100 req/min), 역할 기반 권한을 적용한다. 민감 데이터는 암호화 저장하며, 프롬프트 인젝션 탐지를 위한 필터를 구현한다.

## 5. 평가 지표 및 검증 도구

- **정답 일치도 (Answer Matching)**: TRM/CompassVerifier와 같은 reward model로 생성된 정답과 기준 정답을 비교하여 정확도와 신뢰도를 동시에 평가한다 [17]. 카테고리별로 Surface Accuracy, Semantic Similarity 점수를 산출한다.
- **Semantic Energy Score**: 샘플링 기반으로 문항의 불확실성(내적 에너지) 값을 계산하여 이상치 문항을 검출한다. 임계값은 경험적으로 설정하며 학습 과정에서 업데이트한다.
- **PRM 점수**: Process Reward Model을 활용하여 각 추론 단계의 논리 일관성을 0~10 점수로 매기고 평균값을 사용한다.
- **최종 QA 통과율**: 위 필터를 통과한 문항의 비율을 측정하여 시스템 성능을 추적한다.
- **비용 지표**: 모델별 토큰 사용량과 비용(USD)을 트래킹하며, Best-of-N 및 재평가 파라미터를 조정한다.

## 6. 결론 및 향후 계획

이 설계는 2026 년 현재 사용 가능한 최신 LLM, 에이전트 프레임워크, DSPy 3.0, Next.js 16 등을 기반으로 **영어 시험지 자동 생성 및 검수 시스템** 을 구현하기 위한 구체적인 청사진을 제시한다. GPT-5.2, Gemini 3.1 Pro, Claude 4.6, Llama 4 Scout 등 다양한 모델을 상황에 맞게 라우팅하여 사용하며, DSPy의 선언적 코딩과 자동 최적화로 프롬프트 엔지니어링을 제거한다. Semantic Energy와 PRM을 통해 환각과 논리 결함을 정량적으로 탐지하고, Answer Matching과 TRM/CompassVerifier로 객관식의 복수 정답 문제를 해결한다.

향후에는 **Reinforcement Learning from Human Feedback (RLHF)**, **Active Learning**을 도입하여 데이터 수집 효율을 높이고, **모듈 간 캐시 공유** 및 **온디바이스 모델** 활용을 통해 지연과 비용을 더욱 최적화할 수 있다. 또한 국내 교육 규정과 개인정보 보호법을 준수하는 데 필요한 법적 요건을 점검해야 한다.

---


[1] GPT-5.2 Model | OpenAI API
https://developers.openai.com/api/docs/models/gpt-5.2

[2] Gemini 3.1 Pro - Model Card — Google DeepMind
https://deepmind.google/models/model-cards/gemini-3-1-pro/

[3] What's new in Claude 4.6 - Claude API Docs
https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-6

[4] Unmatched Performance and Efficiency | Llama 4
https://www.llama.com/models/llama-4/

[5] The best AI models in 2026: What model to pick for your use case | Pluralsight
https://www.pluralsight.com/resources/blog/ai-and-data/best-ai-models-2026-list

[6] [7] [8] Top 5 Open-Source Agentic AI Frameworks in 2026
https://aimultiple.com/agentic-frameworks

[9] [10] [11] [12] [13] DSPy 3.0: The Evolution from Prompt Engineering to Programming | by Anubhab Mohanty | Medium
https://devoscientist.medium.com/dspy-3-0-the-evolution-from-prompt-engineering-to-programming-b8df5ead5c60

[14] text-embedding-3-large Model | OpenAI API
https://developers.openai.com/api/docs/models/text-embedding-3-large

[15] AI Model Catalog | Microsoft Foundry Models
https://ai.azure.com/catalog/models/Cohere-embed-v3-multilingual

[16] Next.js 16 | Next.js
https://nextjs.org/blog/next-16

[17] Textual Answer-Matching Reward
https://www.emergentmind.com/topics/textual-answer-matching-reward